Contradiction-Aware Sparse Memory Finetuning for Continual Learning in Large Language Models *

Lydia Chatziioannou Pratyush Lahane Cynthia Liu
Ramya Krishnan Aakriti Shah Athirai Shanmugam Rylan Wade

CSCI 566: Deep Learning and Its Applications – Final Report
University of Southern California

Abstract

Large Language Models (LLMs) perform well across many tasks but remain static after deployment. In dynamic settings, models must incorporate new information without degrading prior knowledge, yet naive fine-tuning leads to catastrophic forgetting. We propose **Contradiction-Aware Sparse Memory (CASM) Finetuning**, a continual learning framework that models knowledge as versioned and time-scoped. CASM isolates updates within a sparse parameter subspace, detects contradictions between new and prior knowledge, and routes queries to the appropriate knowledge version at inference time. Experiments on temporally structured data show that isolating updates in sparse memory significantly improves continual learning performance over standard fine-tuning and LoRA. However, under limited data and compute, a simpler sparse memory baseline (SMF) achieves higher overall stability and consistency than CASM. We hypothesize that routing and slot-based versioning fragment the training signal, limiting effective memory utilization in low-data regimes. These results suggest that while contradiction-aware versioning is a promising direction, its benefits depend on sufficient training signal and effective routing.

1 Introduction & Motivation

Language models encode world knowledge statically at training time, yet the world is constantly changing. Geopolitical borders change, scientific consensus updates, infrastructure specifications change, and organizations evolve, often in ways that directly contradict previously correct truths. A model trained on data through a fixed cutoff date has no mechanism to incorporate these updates without expensive retraining, and even then, naive retraining tends to overwrite prior knowledge rather than extending it. This brittleness with respect to temporal knowledge is not just an inconvenience, it is a fundamental limitation for any deployment where factual accuracy over time is required.

Prior work has approached this problem from several angles. Continual learning methods aim to train models sequentially without catastrophic forgetting, typically through regularization, replay buffers, or architectural isolation of new knowledge. Model editing techniques (e.g. ROME (Meng et al., 2022a), MEMIT (Meng et al., 2022b)) surgically modify specific factual associations in a model's weights. Temporal reasoning work addresses how models should interpret time-scoped queries, and mixture-of-experts architectures provide sparse, parameter-efficient computation through learned routing. Each of these ideas addresses only a slice of the full problem.

No existing framework jointly satisfies the following requirements:

- rehearsal-free continual learning for evolving factual updates
- preservation of multiple coexisting, potentially contradictory knowledge states across time periods
- sparse and parameter-efficient memory updates that avoid touching frozen pretrained weights
- and inference-time routing that can direct a time-scoped query to the appropriate period-specific knowledge

Previous approaches treat these as separate concerns. The result is that a model may handle catastrophic forgetting without managing contradiction, or support temporal reasoning without enabling efficient incremental updates.

We propose CASM (Contradiction-Aware Sparse Memory Finetuning), a framework designed

*Code available at: https://github.com/athirai-s/Continual-Learning

to address the gap directly. CASM augments a frozen LLM with a bank of small, trainable memory slots: lightweight parameter sets that can be updated without modifying the base model weights. A learned router dynamically selects which slots are activated for a given input, based on cosine similarity between the input's embedding and each slot's prototype vector. When a fact changes across time periods, CASM's contradiction detector identifies the conflict and instantiates a new slot with a semantically grounded prototype, allowing both the old and new fact to coexist in separate, independently routable memory locations. At inference time, temporal context in the query steers routing toward the appropriate period's slot, enabling time-scoped knowledge retrieval without modifying the base model architecture.

1.1 Contributions

Our main contributions are as follows:

- We identify and formalize the joint requirement for rehearsal-free continual updates, contradiction preservation, sparse parameter efficiency, and temporal routing (a combination unaddressed by prior work).
- We introduce CASM, a contradiction-aware sparse memory framework that integrates these requirements into a unified architecture layered on frozen LLMs.
- We propose a dynamic slot instantiation mechanism that detects factual contradictions across time periods and creates semantically grounded memory slots to represent each knowledge state.
- We design a prototype-based routing mechanism that uses cosine similarity over period-conditioned embeddings to direct inference-time queries to the correct temporal slot without any retrieval index or external memory store.
- We empirically evaluate CASM on a temporally structured dataset and compare it to existing methods like LoRA, Full FT, and SMF, demonstrating its ability to incorporate updates while preserving historical knowledge, though with lower consistency than simpler sparse memory baselines.

2 Literature Review

2.1 Catastrophic Forgetting in Sequential Learning

Catastrophic forgetting occurs when neural networks trained sequentially on new data degrade in performance on previously learned tasks. Early approaches such as Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017) mitigate forgetting by penalizing changes to parameters deemed important for earlier tasks. While effective in smaller models, such regularization-based methods scale poorly to large language models (LLMs), whose knowledge is highly distributed across parameters.

Rehearsal-based approaches reduce forgetting by replaying stored or synthetic past data. Recent work on self-synthesized rehearsal extends this idea to LLMs, but replay introduces additional computational cost and risks amplifying hallucinated or low-quality samples. In many real-world applications, storing past data may also be infeasible due to privacy or availability constraints. These limitations motivate rehearsal-free continual learning methods.

2.2 Continual Learning in LLMs

Parameter-efficient fine-tuning (PEFT) methods have emerged as promising rehearsal-free strategies. CL-LoRA (He et al., 2025) extends LoRA by allocating task-specific low-rank adapters while preserving shared parameters, reducing destructive interference. Sparse Memory Finetuning (SMF) (Lin et al., 2025) freezes the base model and introduces a small trainable memory in which only a sparse subset of parameters is updated during new learning phases, improving retention while maintaining computational efficiency.

Although these methods mitigate catastrophic forgetting, they primarily focus on task- or domain-incremental benchmarks. They do not explicitly address evolving factual knowledge, where new updates may contradict earlier answers. Standard continual learning metrics evaluate average accuracy and forgetting rates but do not measure internal factual consistency across time.

2.3 Model Editing and Knowledge Overwriting

A related line of work investigates direct factual editing in LLMs. ROME (Meng et al., 2022a) and MEMIT (Meng et al., 2022b) identify and modify

specific parameter subspaces responsible for encoding factual associations, enabling efficient updates without full retraining.

However, these methods are inherently overwriting mechanisms: when a fact changes, the previous association is replaced. In dynamic real-world settings, both historical and current facts may remain valid depending on temporal context (e.g., "Who was the U.S. president in 2010?" vs. "Who is the U.S. president now?"). Existing editing approaches do not preserve multiple coexisting factual states within a single model.

Thus, while model editing enables efficient updates, it does not support versioned or time-scoped knowledge retention.

2.4 Temporal Knowledge and Time-Aware QA

Temporal reasoning has been widely studied in knowledge graph question answering. Work on Complex Temporal Question Answering (Jia et al., 2021) and surveys on Temporal Knowledge Graph QA (Su et al., 2024) emphasize the importance of modeling time-scoped facts and evolving entity attributes. More recent approaches incorporate temporal constraints into LLM-based retrieval pipelines, improving performance on time-sensitive queries.

However, these methods rely on structured external knowledge sources rather than addressing how evolving facts should be represented within parametric LLM memory. LLMs trained on static corpora often exhibit temporal drift, applying outdated knowledge to present queries. No existing approach explicitly integrates time-scoped fact versioning into rehearsal-free parametric continual learning.

This gap highlights the need for mechanisms that store and retrieve versioned knowledge directly within the model.

2.5 Sparse Routing and Modular Specialization

Sparse activation architectures such as Switch Transformers (Fedus et al., 2022) demonstrate that routing inputs to selective parameter subsets enables specialization and reduces interference. AdapterFusion (Pfeiffer et al., 2021) similarly allows non-destructive task composition.

These works suggest that selective parameter activation can preserve knowledge across tasks. However, routing has not been used to disambiguate coexisting contradictory factual states or to enable inference-time selection of time-specific knowledge modules.

2.6 Contradiction Detection and Factual Consistency

Natural Language Inference (NLI) datasets such as SNLI (Bowman et al., 2015) provide foundational methods for detecting entailment and contradiction between statements. Benchmarks like TruthfulQA (Lin et al., 2022) demonstrate that LLMs frequently generate factually incorrect or inconsistent answers.

Despite advances in factual evaluation, existing continual learning approaches do not explicitly identify when newly introduced knowledge contradicts prior model outputs. Without contradiction-aware mechanisms, sequential updates may produce internal inconsistencies or hybridized answers.

3 Proposed Approach

CASM-Finetuning is built on a shared continual-learning infrastructure that implements four training methods under identical training, checkpointing, and routing conditions, so that observed differences primarily reflect method design, though CASM hyperparameters were varied across scales to explore a broader design space (see Section 7.3). The four methods form a progression of increasing version awareness: full fine-tuning (`full_ft`) updates every backbone parameter; LoRA adaptation (`lora`) restricts updates to low-rank adapters; Sparse Memory Finetuning (`smf`) freezes the backbone and trains a single sparse memory; and our proposed method, Contradiction-Aware Sparse Memory Finetuning (`casm`), replaces that single memory with a routed bank of *versioned* slots. CASM is the only method in this comparison explicitly designed to maintain coexisting factual versions and retrieve them at inference time.

We initially explored TSQA and TGQA as continual-learning targets but discontinued both: the question–answer surface occluded the underlying fact transitions, and probe coverage was too sparse to drive a learned router. We instead use TemporalWiki (Jang et al., 2022) together with a synthetic companion benchmark; both datasets, the model scales we evaluate, and the per-method hyperparameters are described in Section 4.

3.1 Shared Continual Learning Foundation

A single runner (`train_runner.py`) orchestrates all four methods. It iterates over temporal periods, calls `trainer.train_period()` for each, checkpoints after every period, and optionally evaluates between periods. The runner is deliberately method-agnostic: behavioral differences between methods are localized to the model wrapper and trainer branches, not the orchestration loop. A single `TrainConfig` object specifies the active method, precision, dataset, batching, and method-specific hyperparameters, so that one configuration switch determines whether a run is full fine-tuning, LoRA, SMF, or CASM. This shared foundation is what makes a controlled four-way comparison possible in the first place.

3.2 Sparse Memory Finetuning (SMF)

SMF is our intermediate baseline between parameter-efficient adaptation and version-aware continual learning. Its purpose is diagnostic: SMF isolates the contribution of *constrained parameter updates* alone, so that any additional gain from CASM can be attributed specifically to versioning and routing rather than to sparsity in general.

3.2.1 Architecture

SMF wraps the base LLM and freezes all backbone parameters. A small trainable sparse-memory block is attached to a configurable set of layers (`smf_update_layers`). Updates flow through a learned sparse gate that controls which memory dimensions are active, enforcing that only a fraction (`smf_sparsity_ratio`) of memory parameters contribute at any given step. The memory contribution is additive to the frozen backbone's output, keeping the base model intact. A single shared memory state is maintained across all training periods; there is no routing or versioning, by design.

3.2.2 Training Objective

The optimizer is restricted to SMF parameters only, excluding all frozen backbone weights. The training objective combines a standard causal language-modeling loss with an optional sparsity regularization term that encourages compact memory usage:

$$\mathcal{L}_{\text{SMF}} = \mathcal{L}_{\text{task}} + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{sparse}}, \qquad (1)$$

where λ_{reg} is controlled by `smf_regularization_weight`.

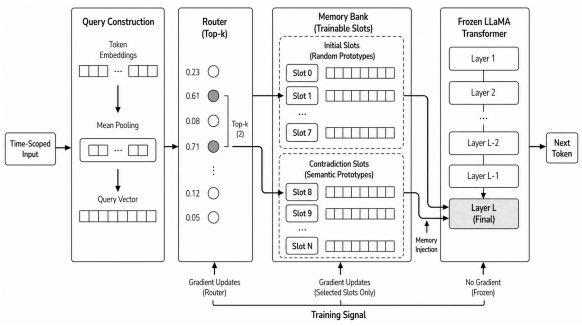

Figure 1: Overview of the CASM framework, including versioned memory slots, contradiction detection, and inference-time routing.

3.3 Contradiction-Aware Sparse Memory Finetuning (CASM)

CASM addresses the failure mode SMF leaves untouched (Figure 1): when a fact changes between periods, SMF's single shared memory must overwrite the prior value to absorb the new one, destroying the older version in the process. CASM extends SMF along three coordinated axes that together eliminate this trade-off: a *versioned memory bank* replaces the single shared memory; a *contradiction detector* decides when the bank should branch rather than overwrite; and a *router* performs inference-time slot selection so that the correct version is retrieved for the current query. The three components are interdependent — a versioned bank without a router is unreachable storage, and a router without a contradiction signal has nothing temporally meaningful to route over.

3.3.1 Versioned Memory Bank

The single SMF memory block is replaced by a bank of memory slots. Each slot holds trainable parameters along with metadata: subject, relation, `valid_from`, `valid_until`, usage counts, and links to parent and contradicting slots. Slots are never silently overwritten. When a fact changes, the prior slot is closed and a new slot is created with the new value, forming a version chain that captures the full history of factual states. Closed slots remain in storage and remain queryable, so retrieving a historical fact remains possible even after the fact has been updated in a later period.

3.3.2 Contradiction Detection

At the start of each training period the system loads the period's passages and identifies *changed probes*—facts that differ from the current registry state. These probes are passed through a

`ContradictionDetector` that compares new evidence against registered facts. The detector's output is actionable rather than passive: it determines the memory update path. If a contradiction is detected, a new versioned slot is created and linked to the prior version. If no contradiction is detected, an existing slot is updated in place. This makes contradiction detection a first-class part of the training protocol — the mechanism that decides *how* memory is written — rather than a post-hoc metric.

3.3.3 Temporal Router

A lightweight router accepts a compact query representation and an optional temporal signal, and emits top-k slot ids together with softmax routing weights. The router is trained jointly with the slot memories so that selection and storage co-adapt: slots cannot drift into representations the router cannot recover, and the router cannot develop preferences for slots whose contents do not exist yet. At inference time, the router selects the slot(s) most appropriate for the query's temporal context, enabling the model to retrieve historical facts without overwriting them. Router behavior is configurable through `casm_top_k` and `casm_router_temperature`.

3.3.4 Training Protocol

Within `CASFTrainer.train_period()`, the CASM branch executes the following sequence for each period:

1. Load the period's passages.
2. Collect changed probes against the current registry.
3. Run `detector.check(probes, registry)` to identify contradictions.
4. Route each training example to the appropriate slot(s).
5. Create new slots for contradicted facts; update existing slots otherwise.
6. Train the selected slots and the router while keeping non-selected slots frozen.
7. Write updated facts back into the registry.

3.3.5 Loss Composition

CASM optimizes a composite objective in which each term targets a distinct failure mode the design must avoid:

$$\mathcal{L}_{\text{CASM}} = \mathcal{L}_{\text{task}} + \alpha \cdot \mathcal{L}_{\text{router}} + \beta \cdot \mathcal{L}_{\text{sparse}} + \gamma \cdot \mathcal{L}_{\text{overlap}}. \quad (2)$$

$\mathcal{L}_{\text{task}}$ preserves language-modeling quality on the current period's passages. $\mathcal{L}_{\text{router}}$ trains slot selection so that the right version is retrieved at inference. $\mathcal{L}_{\text{sparse}}$ enforces focused per-slot updates, preventing slots from becoming dense and entangled. $\mathcal{L}_{\text{overlap}}$ penalizes distinct slots from collapsing to similar behavior, which would undermine the versioning the memory bank is intended to provide. Each term is independently configurable and logged separately during training.

3.3.6 Checkpointing

CASM's state extends beyond the model weights, so its checkpoints store the full `MemoryRegistry` (slot metadata, slot parameters, and version links), the router state, and the contradiction linkages, alongside the standard backbone weights, tokenizer, configuration, and period marker. Checkpoint compatibility is validated strictly before resume; mismatched checkpoint–method combinations are rejected to prevent silent corruption of the version graph.

4 Experimental Setup

4.1 Datasets

We evaluate continual factual learning using a temporally ordered subset of TemporalWiki together with a controlled companion benchmark, Synthetic TemporalWiki. The TemporalWiki (Jang et al., 2022) subset is organized into four sequential periods:aug_sep, sep_oct, oct_nov, and nov_dec. Each period contains facts that either change relative to earlier periods or remain unchanged, enabling direct evaluation of both factual updating and retention.

We also use Synthetic TemporalWiki, a fully controlled diagnostic benchmark generated using the Gemini API on entirely fictional entities. The primary motivation for its construction was computational: the full TemporalWiki corpus spans roughly 800,000 Wikipedia pages per period, far exceeding our available training budget. A smaller, controlled alternative was necessary to make continual learning experiments tractable.

Facts are drawn from eight thematic domains (maritime, aerospace, municipal, mining, archival, agricultural, diplomatic, and infrastructure) and organized as (entity, relation) pairs with values defined across four time periods: 2018, 2020, 2022, and 2024. No real-world people, places, or organizations appear in the dataset. The dataset comprises

Method	Backbone	Memory	Versioning	Routing
Full FT	Trainable	—	No	No
LoRA	Frozen	Low-rank	No	No
SMF	Frozen	Sparse (1)	No	No
CASM	Frozen	Sparse (n slots)	Yes	Yes

Table 1: Comparison of training methods along four axes. CASM is the only method that supports coexisting fact versions and inference-time temporal routing.

605 facts in total, with change events distributed roughly evenly across the three period boundaries, yielding approximately 150 contradictions per transition. Fact values are constrained to four words or fewer to ensure unambiguous ground truth at evaluation time.

Training passages are generated by the Gemini API as short one-to-three sentence augmentations, with each passage required to state the target fact exactly once; remaining sentences supply brief topical context. Surface-form diversity is introduced by randomizing generation temperature per call. In this way, Synthetic TemporalWiki trades the realistic temporal drift of the full corpus for fully controlled ground truth over when facts change and when historical versions should remain distinct.

4.2 Models

We evaluate all methods on Llama-based instruction-tuned backbones at two scales: 1B and 3B. The 1B model is used for baseline comparisons and ablation studies, while the 3B model is used to verify that observed trends hold at a larger scale. All models use the same tokenizer and are loaded in mixed precision, using bf16 for 1B experiments and fp16 for 3B experiments.

4.3 Baselines and Compared Methods

We compare four methods within the same training and evaluation pipeline: Full Fine-Tuning, LoRA, SMF, and CASM.

Full Fine-Tuning (`full_ft`) updates all backbone parameters sequentially across temporal periods. This baseline offers the highest adaptation capacity but is also the most vulnerable to catastrophic forgetting, since new updates can overwrite previously learned knowledge.

LoRA (`lora`) provides a parameter-efficient baseline in which the backbone is frozen and only low-rank adapters are trained. We apply LoRA to the attention projection modules (`q_proj`, `k_proj`, `v_proj`, `o_proj`), allowing the model to adapt to new temporal information without modifying the full parameter set.

SMF (`smf`) uses a frozen backbone together with a single shared sparse trainable memory. This method isolates the effect of sparse parameter updates while omitting any explicit versioning or routing mechanism. It therefore serves as a useful intermediate baseline between standard parameter-efficient adaptation and full version-aware continual learning.

CASM (`casm`) extends the sparse-memory setting by introducing multiple memory slots, contradiction-aware version branching, and inference-time routing. The backbone remains frozen, but updates are written into slot-specific sparse memory states rather than a single shared memory bank. CASM is the only method in our comparison explicitly designed to preserve and retrieve coexisting factual versions.

4.4 Training Configuration

Training hyperparameters for all methods at both scales are reported in Table 2. The 1B and 3B models share the same per-period epoch count and batch size but differ in learning rate, gradient accumulation steps, and floating-point precision. Training proceeds sequentially over temporal periods in chronological order, with checkpoints saved after each period.

For `full_ft`, all model parameters are updated at every period. For `lora`, only the adapter parameters are optimized while the backbone remains frozen; adapters are applied to the attention projection modules. For `smf`, only the sparse memory parameters are updated; update layers are chosen to target mid-to-late transformer layers and differ by model depth. For `casm`, additional hyperparameters control the number of memory slots, router type and temperature, top-k slot selection, and the weights on sparsity and anti-overlap regularization. At the 1B scale, CASM uses a similarity-based router with period-deterministic slot assignment (8 slots per period, 32 total); at the 3B scale, a learned MLP router is used with a smaller slot bank of 6 slots.

Because CASM maintains state beyond back-

bone weights, its checkpoints additionally store the full memory registry, including slot metadata, slot parameters, version links, router state, and contradiction linkages. Checkpoint compatibility is validated strictly before resume to prevent silent corruption of the version graph.

4.5 Loss Functions

For `full_ft`, `lora`, and `smf`, training uses the standard task loss on the current period's data.

`casm` is optimized with a composite objective:

$$\mathcal{L}_{\text{CASM}} = \mathcal{L}_{\text{task}} + \alpha\mathcal{L}_{\text{router}} + \beta\mathcal{L}_{\text{sparse}} + \gamma\mathcal{L}_{\text{overlap}}.$$

Here, $\mathcal{L}_{\text{task}}$ preserves language modeling quality on the current period, $\mathcal{L}_{\text{router}}$ trains slot selection, $\mathcal{L}_{\text{sparse}}$ encourages focused sparse updates, and $\mathcal{L}_{\text{overlap}}$ discourages distinct slots from collapsing to similar behavior. Each term is configured independently and logged separately during training.

4.6 Evaluation Metrics

We report continual-learning metrics after every temporal period rather than only at the end of training. The primary metrics are:

- **Plasticity**: performance on the current period's changed probes after training on that period.
- **Stability**: performance on unchanged probes from earlier periods after continued training.
- **Forgetting**: the drop in retained performance on previously learned facts after later updates.

Reporting metrics per period allows us to analyze how each method balances adaptation to newly changed facts against preservation of previously learned knowledge throughout the continual-learning trajectory.

5 Results

We report results on the Synthetic TemporalWiki benchmark across four temporal periods (2018, 2020, 2022, 2024) at both 1B and 3B model scales. As described in Section 4.1, Synthetic TemporalWiki was constructed as a compute-tractable alternative to the full TemporalWiki corpus. All metrics are computed per period over changed and unchanged probe sets separately, as described in Section 4.6. Table 3 reports averages across periods 2020–2024; period 2018 is excluded from aggregation as no prior period exists from which retention can be measured. Full per-period results are provided in Appendix Table 4.

Across all methods, the 3B model showed uniformly weak performance across all methods, with plasticity scores near zero on changed probes and only marginal stability accumulation on unchanged probes; none of the methods diverged meaningfully from one another at this scale. We attribute this to the scale mismatch between the 3B backbone and our training budget: the synthetic dataset of 605 facts and limited compute were insufficient to meaningfully shift a higher-capacity model. We therefore focus our analysis on the 1B results, which exhibit clear differentiation across methods and offer the more informative comparison.

5.1 Plasticity on Changed Probes

Across periods, SMF and CASM consistently outperform Full FT and LoRA on plasticity, confirming that sparse memory updates provide a more effective mechanism for absorbing new factual information than either full parameter updates or low-rank adaptation.

SMF achieves the strongest average plasticity (0.591), peaking at 0.667 in the 2022 period and sustaining high performance through 2024 (0.533). CASM is competitive at select periods, reaching 0.680 in 2022 — the highest single plasticity score across all methods and periods — but is inconsistent across the full trajectory, falling to 0.127 in 2020 and 0.180 in 2024, yielding an average of 0.329. Full FT and LoRA lag considerably, with LoRA peaking at 0.173 and Full FT reaching at most 0.113.

5.2 Stability on Unchanged Probes

Stability scores are zero for all methods in the 2018 period, which is expected: with no prior training period, there are no previously learned facts to retain. From 2020 onward, stability diverges meaningfully across methods.

SMF accumulates the strongest stability over time, reaching 0.715 by the 2024 period — the highest stability score observed across any method. LoRA also accumulates substantial stability (average 0.221), while CASM achieves higher average stability (0.258) but still trails SMF. Notably, SMF's stability begins near zero in 2020 (0.064) before rising sharply, while CASM starts higher (0.193) but plateaus more quickly. Full FT shows limited stability retention (average 0.139), substantially lower than all sparse memory approaches.

Category	Hyperparameter	1B Model	3B Model
Shared	Learning Rate	5×10^{-4}	2×10^{-4}
	Batch Size	1	1
	Gradient Accum.	4	16
	Epochs per Period	20	20
	Precision	bf16	fp16
	Random Seed	42	42
LoRA	Rank (r)	16	16
	Alpha	32	32
	Dropout	0.05	0.05
	Target Modules	`q_proj, k_proj, v_proj, o_proj`	
SMF	Memory Size	64	64
	Sparsity Ratio	0.1	0.1
	Update Layers	4, 8, 12	7, 14, 21
	Regularization Weight	0.01	0.01
CASM	Num Slots	32	6
	Slots per Period	8	—
	Memory Size	64	512
	Router Hidden Size	256	512
	Router Type	similarity	mlp
	Top-k	2	1
	Router Temperature	1.0	0.3
	Sparsity Weight (β)	0.01	0.001
	Overlap Weight (γ)	0.01	0.001

Table 2: Hyperparameters for all methods at 1B and 3B scale. Shared hyperparameters apply to all four methods. Method-specific hyperparameters are only active for the indicated method. SMF update layers differ by scale to reflect the differing depth of the 1B (16 layers) and 3B (28 layers) backbones.

	1B			3B		
Method	Plasticity	Stability	Token F1	Plasticity	Stability	Token F1
Full FT	0.089	0.139	0.048	0.082	0.143	0.027
LoRA	0.138	0.221	0.097	0.096	0.114	0.000
SMF	0.591	0.482	0.112	0.118	0.215	0.044
CASM	0.329	0.258	0.056	0.072	0.083	0.024

Table 3: Average plasticity, stability, and token F1 on Synthetic TemporalWiki across periods 2020–2024 at 1B and 3B scale. Token F1 is averaged across changed and unchanged probes. Period 2018 is excluded as no prior period exists from which retention can be measured.

The gap between SMF and CASM on stability is notable: while CASM's versioned memory design is intended to preserve prior knowledge through slot isolation, its stability on unchanged probes falls short of SMF's single shared memory across all periods, suggesting that the routing and branching mechanisms require further tuning to fully realize their retention benefits.

Stability on changed probes is uniformly zero for all methods at all periods. This is structurally expected: changed probes require the model to update away from prior answers, so any match against the prior answer registers as zero. This reflects the design of the evaluation protocol rather than a failure of retention.

5.3 Token F1

Token F1 scores are low across all methods and periods, consistent with the difficulty of open-ended factual completion on entirely fictional entities absent from the base model's pretraining distribution. Relative differences nonetheless remain informative. SMF leads on both changed probes (peak 0.114 at 2022) and unchanged probes (peak 0.122 at 2024), with LoRA achieving the highest unchanged token F1 by 2024 (0.147). CASM shows modest but consistent token F1 on changed probes throughout. The generally low token F1 across all methods suggests that while probe-level exact match captures factual updating, surface-level generation quality on novel fictional entities remains a persistent challenge independent of training method.

6 Discussion

Results at 1B scale show clear separation between methods. SMF achieves the highest average plasticity (0.591) and stability (0.482), with both metrics increasing across periods. CASM achieves competitive peak plasticity (0.680 in 2022) but lower aver-

age performance (plasticity 0.329, stability 0.258) and greater variability across periods. Full fine-tuning and LoRA remain low across all metrics, indicating limited ability to incorporate new information or retain prior knowledge.

The gap between SMF and CASM indicates that adding versioned memory and routing does not improve performance under the current training conditions. We hypothesize that routing errors and slot under-utilization fragment the training signal, preventing sufficient parameter updates within each memory slot. In contrast, SMF's single shared memory concentrates updates into a unified parameter space, enabling more efficient learning and retention in low-data regimes.

At 3B scale, all methods show minimal learning signal, with near-zero plasticity and limited stability accumulation. This suggests that the dataset size (605 facts) and training budget are insufficient to meaningfully update a higher-capacity model. As a result, differences between methods are most informative at 1B scale.

Token F1 remains low across all methods and periods, consistent with the use of synthetic entities not present in pretraining. While exact-match metrics capture whether the correct fact is retrieved, low token F1 indicates that models struggle to reliably generate those facts in free-form output.

Overall, results indicate that sparse memory updates are the primary driver of continual learning performance in this regime. While CASM provides a mechanism for contradiction-aware versioning, its benefits are not realized under limited data and compute, and are likely to depend on improved routing and larger-scale training settings.

7 Conclusion

We introduced Contradiction-Aware Sparse Memory (CASM) Finetuning, a continual learning framework that combines rehearsal-free factual updates, versioned memory slots, sparse parameter-efficient training, and inference-time temporal routing layered on a frozen language model. We evaluate it alongside full fine-tuning, LoRA, and SMF baselines on temporally structured updates, measuring plasticity and stability across sequential periods. Across all methods, isolating updates from the backbone improves performance: SMF and CASM consistently outperform full fine-tuning and LoRA on changed probes, indicating that sparse memory updates are more effective for incorporating new facts than full or low-rank parameter updates.

SMF achieves the strongest overall performance, with the highest average plasticity (0.591) and stability (0.482) at 1B scale. Performance remains consistent across periods, with both metrics increasing over time. These results indicate that a single shared sparse memory is sufficient to support both adaptation and retention under the current data regime.

CASM introduces versioned memory slots and inference-time routing, enabling explicit representation of coexisting factual states. CASM achieves competitive peak plasticity (0.680 in 2022), but performance is inconsistent across periods, yielding lower average plasticity (0.329) and stability (0.258) than SMF. This gap suggests that the additional routing and slot-branching components introduce optimization challenges that limit effective memory utilization.

Full fine-tuning and LoRA underperform across all metrics, with low plasticity and limited stability accumulation, confirming that direct parameter updates are insufficient for continual factual learning in this setting.

Overall, results indicate that constrained memory updates are the dominant factor for continual learning performance under limited data and compute. While CASM provides a mechanism for contradiction-aware versioning, its benefits are not realized under the current training conditions. Further work is required to improve routing, slot utilization, and training dynamics, particularly in higher-data regimes where explicit versioning may offer greater advantages.

8 Limitations & Future Directions

While CASM introduces a principled framework for contradiction-aware continual learning, our empirical results reveal several limitations that point to possible future directions. Given more time and computational resources:

8.1 Compute and Data Scale

The most immediate limitation is the mismatch between CASM's architectural complexity and our available training budget. The Synthetic TemporalWiki benchmark, while useful for controlled evaluation, comprises only 605 facts (a corpus too small to fully exercise a versioned memory bank with learned routing). SMF's simpler single-memory design is naturally better suited to this regime, as

it has fewer components to optimize jointly. We expect CASM's versioning and routing machinery to show stronger relative gains as the number of facts, contradiction events, and temporal periods grows. Evaluating on the full TemporalWiki corpus or temporally structured benchmarks with adequate compute would be a natural next step.

8.2 Router and Slot Optimization

CASM's stability gap relative to SMF suggests that the router and slot-branching components introduce optimization challenges that are not yet fully resolved. In particular, the anti-overlap loss intended to keep slots distinct may conflict with the task loss in low-data situations, causing this instability. Future work should investigate strategies that phase in router training after slots have been seeded with sufficient factual signal, as well as initialization for slot prototypes based on embeddings rather than random vectors.

8.3 Hyperparameter Inconsistency Across Scales

The CASM configurations at 1B and 3B scale were deliberately varied to explore a broader region of the design space within our available compute budget. Rather than fixing all hyperparameters and varying only model size, we used the two scales to investigate different routing strategies: the 1B model uses a cosine similarity router with period-deterministic slot assignment (8 slots per period, 32 total), while the 3B model uses a learned MLP router with a smaller slot bank of 6 slots. Memory size, router temperature, and regularization weights also differ between scales. Future work should establish a shared CASM configuration that varies only model size across scales, enabling a cleaner analysis of how backbone capacity interacts with versioned memory and routing.

9 Computational Resources

We report the computational resources used across all experiments and dataset construction.

Training Compute. All 1B experiments were conducted on Google Colab using NVIDIA A100 GPUs, consuming approximately 200 A100 GPU-hours across all methods and temporal periods. All 3B experiments were conducted on the University of Southern California's CARC (Center for Advanced Research Computing) cluster, consuming approximately 300 A100 GPU-hours across all methods and temporal periods.

Synthetic Dataset Generation. The Synthetic TemporalWiki benchmark was generated using the Gemini API. Dataset construction consumed approximately 1,494,000 input tokens and 422,000 output tokens across all fact generation and passage augmentation calls.

Frameworks. All models were implemented in PyTorch using the HuggingFace Transformers and PEFT libraries. Tokenization and model loading used the standard HuggingFace model hub infrastructure.

A Per-Period Results on Synthetic TemporalWiki

Table 4 reports plasticity, stability, and token F1 for all four methods at the 1B scale across all four temporal periods of Synthetic TemporalWiki.

Period	Condition	Full FT	LoRA	SMF	CASM
Plasticity (changed probes)					
2018	changed	0.000	0.000	0.000	0.000
2020	changed	0.113	0.147	0.573	0.127
2022	changed	0.081	0.173	0.667	0.680
2024	changed	0.073	0.093	0.533	0.180
Stability (unchanged probes)					
2018	unchanged	0.000	0.000	0.000	0.000
2020	unchanged	0.143	0.224	0.064	0.193
2022	unchanged	0.145	0.224	0.667	0.283
2024	unchanged	0.130	0.215	0.715	0.297
Token F1 (changed probes)					
2018	changed	0.036	0.017	0.088	0.036
2020	changed	0.056	0.058	0.114	0.043
2022	changed	0.043	0.094	0.114	0.058
2024	changed	0.041	0.080	0.105	0.050
Token F1 (unchanged probes)					
2018	unchanged	0.000	0.000	0.000	0.000
2020	unchanged	0.045	0.078	0.107	0.047
2022	unchanged	0.049	0.124	0.107	0.068
2024	unchanged	0.051	0.147	0.122	0.073

Table 4: Per-period plasticity, stability, and token F1 for all methods at 1B scale on Synthetic TemporalWiki. Period 2018 reflects initial seeding only; no prior period exists from which retention can be measured, so plasticity and stability scores of 0.000 at this period are expected by construction.

B Contributions of Each Team Member

- **Lydia Chatziioannou:** Led early project formulation and initial proposal development. Conducted literature review to inform the research direction. Supported the presentation by creating slides and visual assets for both the report and presentation.

- **Pratyush Lahane:** Ran initial continual-learning experiments on the TSQA and TGQA

datasets before the team's pivot to TemporalWiki and synthetic data. Co-defined the architecture of the CASM training pipeline. Ran LoRA experiments on Llama 3.2 1B in Google Colab. Made adjustments to the synthetic-data slot-assignment logic and authored an updated synthetic CASM training script. Presented slides of the project presentation.

- **Cynthia Liu:** Improved the reliability of the training and evaluation pipeline by fixing experiment-protocol, checkpointing, and evaluation issues that affected forgetting measurements. Added regression tests and safeguards for cleaner continual-learning runs. Ran 1B CASM experiments in Colab on the original dataset for metrics and evaluation, and also experimented with a smaller Qwen 0.5B model using SMF.

- **Ramya Krishnan:** Ran continual learning experiments for LoRA, SMF, and CASM on Llama 3.2 3B across all temporal periods on USC's CARC cluster on both original and synthetic datasets. Conducted hyperparameter tuning with varying learning rates to optimize training stability. Performed 1B model evaluation in the sequential continual learning pipeline.

- **Aakriti Shah:** Led evaluation and metrics development, implementing evaluation pipelines and metric reporting scripts for systematic model assessment. Ran Colab training experiments on the Llama 3.2 1B model for metrics and evaluation, running both SMF and Full FT on the original dataset, and SMF and LoRA on the synthetic dataset. Prepared the project presentation slides.

- **Athirai Shanmugam:** Built and ran the continual learning training and evaluation pipeline on USC's CARC cluster. Implemented the pretraining script for the first temporal period, the 3B fine-tuning baseline, and the continual-learning runs that train full fine-tuning, LoRA, SMF, and CASM on the remaining three periods of Llama 3.2 3B. Built the SMF evaluation pipeline and ran full fine tuning on Llama 3.2 1B on the augmented dataset.

- **Rylan Wade:** Led CASM model development, implementing the memory registry, versioned slot routing, contradiction detection, trainer integration, and checkpointing. Drove iterative architectural improvements through multiple model versions, built the synthetic data generation pipeline, designed the period-deterministic slot assignment scheme, and created the Colab training notebooks. Presented slides at the project presentation. Authored key project documentation and resolved numerous training pipeline bugs throughout the project.

References

Samuel Bowman, Gabor Angeli, Christopher Potts, and Christopher D Manning. 2015. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 632–642.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Jiangpeng He, Zhihao Duan, and Fengqing Zhu. 2025. Cl-lora: Continual low-rank adaptation for rehearsal-free class-incremental learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 30534–30544.

Joel Jang, Seonghyeon Ye, Changho Lee, Sohee Yang, Joongbo Shin, Janghoon Han, Gyeonghun Kim, and Minjoon Seo. 2022. TemporalWiki: A lifelong benchmark for training and evaluating ever-evolving language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6237–6250, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Zhen Jia, Soumajit Pramanik, Rishiraj Saha Roy, and Gerhard Weikum. 2021. Complex temporal question answering on knowledge graphs. In *Proceedings of the 30th ACM international conference on information & knowledge management*, pages 792–802.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, and 1 others. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526.

Jessy Lin, Luke Zettlemoyer, Gargi Ghosh, Wen-Tau Yih, Aram Markosyan, Vincent-Pierre Berges,

and Barlas Oğuz. 2025. Continual learning via sparse memory finetuning. *arXiv preprint arXiv:2510.15103*.

Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 3214–3252.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022a. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2022b. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*.

Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. 2021. Adapterfusion: Non-destructive task composition for transfer learning. In *Proceedings of the 16th conference of the European chapter of the association for computational linguistics: main volume*, pages 487–503.

Miao Su, Zixuan Li, Zhuo Chen, Long Bai, Xiaolong Jin, and Jiafeng Guo. 2024. Temporal knowledge graph question answering: A survey. *arXiv preprint arXiv:2406.14191*.